# Experiment No. 5

**Title**: Implementation of a Variational Autoencoder (VAE) on MNIST for New Image Generation

**Aim**: To implement a Variational Autoencoder (VAE) using TensorFlow/Keras, train it on the MNIST handwritten digit dataset, and generate new MNIST-like images using the learned latent space and trained Decoder.

**Objectives**:

1. To understand the architecture and working principle of a Variational Autoencoder (VAE).
2. To implement the Encoder, latent-space sampling using the Reparameterization Trick, and Decoder using TensorFlow/Keras.
3. To train a VAE on the MNIST dataset using Reconstruction Loss and KL Divergence.
4. To visualize reconstructed images, the 2-dimensional latent space, and new images generated from random latent vectors.

**Pre-Requisites**:

1. Basic knowledge of Python programming.
2. Basic understanding of Deep Learning, neural networks and TensorFlow/Keras.
3. Basic understanding of probability and Gaussian distributions.
4. Basic familiarity with the MNIST dataset and image preprocessing.
5. Basic knowledge of Google Colab and GPU execution is desirable.

**Theory**:

Variational Autoencoder: A Variational Autoencoder (VAE) is a Generative AI model that learns a structured latent representation of input data. Unlike a traditional Autoencoder, a VAE learns a probability distribution in the latent space and samples latent vectors from that distribution.

VAE Architecture: The VAE used in this experiment consists of an Encoder and a Decoder. The Encoder converts a 28×28 MNIST image into two outputs: $\mu$ (z_mean) and $\log(\sigma^2)$ (z_log_var). A latent vector z is sampled using the Reparameterization Trick. The Decoder then converts z back into a 28×28 image.

Latent Space: The experiment uses a 2-dimensional latent space. Therefore, each MNIST image is represented by two latent values, z1 and z2. This makes the learned latent space easy to visualize as a two-dimensional scatter plot.

Reparameterization Trick: The VAE samples a latent vector using $z = \mu + \sigma\varepsilon$, where $\varepsilon$ is random noise sampled from a standard normal distribution. Since the Encoder produces $\log(\sigma^2)$, the standard deviation is calculated as $\sigma = \exp(0.5 \log(\sigma^2))$. This formulation allows gradient-based learning through the sampling operation.

VAE Loss: The total VAE loss consists of Reconstruction Loss and KL Divergence. Reconstruction Loss measures how closely the reconstructed image matches the original image. KL Divergence encourages the learned latent distribution to remain close to the standard normal distribution N(0, I). Thus, L_VAE = L_Reconstruction + L_KL.

Generation using the Decoder: After training, a new image can be generated without providing an input image. A random latent vector z is sampled from N(0, I) and passed directly to the trained Decoder. The Decoder converts this latent vector into a new MNIST-like handwritten digit.

**Tools and Technologies Required**

1. Google Colab
2. Python 3
3. TensorFlow 2 / Keras
4. NumPy
5. Matplotlib
6. MNIST dataset
7. GPU support in Google Colab (recommended)
8. Flask, HTML/CSS and JavaScript for the optional deployment extension

**Procedure**:

1. Open Google Colab and select a GPU runtime.
2. Import NumPy, Matplotlib, TensorFlow and Keras libraries.
3. Load the MNIST dataset using keras.datasets.mnist.load_data().
4. Normalize the pixel values from the range 0–255 to the range 0–1.
5. Add the channel dimension so that the image shape becomes 28×28×1.
6. Display sample MNIST images to understand the input data.
7. Set the VAE configuration: original dimension 784, latent dimension 2, batch size 128 and 30 epochs.
8. Build the Encoder using Flatten and Dense layers.
9. Create two Encoder outputs: z_mean (μ) and z_log_var (log(σ²)).
10. Implement the Reparameterization Trick to sample the latent vector z.
11. Build the Decoder using Dense layers followed by Reshape to reconstruct a 28×28×1 image.
12. Create the VAE model with a custom train_step().
13. Calculate Reconstruction Loss and KL Divergence and combine them to obtain the total VAE loss.
14. Train the VAE using the MNIST training images.
15. Plot total VAE loss, Reconstruction Loss and KL Loss across epochs.
16. Use the Encoder and Decoder to reconstruct test images and compare original and reconstructed images.
17. Visualize the 2-dimensional latent space using z_mean values.

18. Generate new images by sampling random latent vectors from N(0, I) and passing them directly to the trained Decoder.
19. Generate a single new image from a random latent vector.
20. Save the trained Encoder and Decoder models in Keras format.
21. Load the saved Decoder and verify that it can generate a new image.
22. Create a ZIP archive containing the saved models for later use.

**Conclusion**:

The experiment successfully demonstrated the implementation of a Variational Autoencoder (VAE) on the MNIST dataset using TensorFlow/Keras. Students learned how the Encoder maps an image to a probabilistic latent representation, how the Reparameterization Trick enables sampling, and how the Decoder reconstructs images. Reconstruction Loss and KL Divergence were used to train the VAE. The 2-dimensional latent space was visualized, and new MNIST-like images were generated by passing random latent vectors to the trained Decoder. The saved Decoder also provides a foundation for developing a Flask-based image-generation application.